

Gestión de Proyectos de Inteligencia Artificial

Actividad Final Fase 3

Objetivo de la actividad:

Documentar la arquitectura, operación, monitoreo, automatización, versionado y mantenimiento de una solución de Inteligencia Artificial basada en prácticas de MLOps y GitOps, integrando un pipeline automatizado mediante GitHub Actions para garantizar la integración continua, trazabilidad y supervisión del ciclo de vida del modelo.

Alumno:

Jonathan Daniel Bernal Sánchez

Matrícula:

T07139185

Institución:

Tecmilenio Campus Cuautitlán

Programa:

Master en Inteligencia Artificial

Profesor:

Luis Ariel Vázquez Piña

Fecha de entrega:

01/08/2026

Lugar:

Estado de México, México

1. Definición de indicadores

Para supervisar el comportamiento del modelo de Machine Learning se definieron indicadores relacionados con su desempeño predictivo, la estabilidad de los datos y el funcionamiento general del sistema de monitoreo.

Los indicadores permiten identificar desviaciones en el comportamiento esperado del modelo, generar alertas y establecer acciones de respuesta ante posibles incidentes.

1.1 Métricas de desempeño del modelo

Accuracy

La exactitud representa la proporción total de predicciones correctas realizadas por el modelo respecto al número total de observaciones evaluadas.

Fórmula:

$$\text{Accuracy} = \text{Predicciones correctas} / \text{Total de predicciones}$$

Esta métrica permite conocer el desempeño general del modelo; sin embargo, debe analizarse junto con otras métricas para evitar interpretaciones incorrectas cuando existe desbalance entre las clases.

Precision

La precisión indica qué proporción de las observaciones clasificadas como positivas fue identificada correctamente.

Fórmula:

$$\text{Precision} = \text{Verdaderos positivos} / (\text{Verdaderos positivos} + \text{Falsos positivos})$$

En este proyecto, esta métrica permite evaluar la confiabilidad de las predicciones positivas generadas por el modelo.

Recall

El Recall, también llamado sensibilidad, representa la capacidad del modelo para identificar correctamente los casos positivos reales.

Fórmula:

$$\text{Recall} = \text{Verdaderos positivos} / (\text{Verdaderos positivos} + \text{Falsos negativos})$$

El Recall fue seleccionado como una de las principales métricas de monitoreo debido a que una disminución significativa puede indicar que el modelo está dejando de detectar observaciones relevantes.

F1 Score

El F1 Score representa la media armónica entre Precision y Recall.

Fórmula:

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Esta métrica permite evaluar de forma equilibrada el desempeño del modelo, considerando tanto los falsos positivos como los falsos negativos.

1.2 Métricas obtenidas durante el entrenamiento

El modelo Random Forest obtuvo los siguientes resultados:

Accuracy: 97.37 %

Precision: 100.00 %

Recall: 92.86 %

F1 Score: 96.30 %

Estos valores representan la línea base de desempeño que será utilizada para comparar futuras ejecuciones del modelo.

1.3 Indicadores de Data Drift

Para detectar cambios en la distribución de los datos se utiliza la prueba estadística Kolmogorov-Smirnov.

El sistema compara la distribución de los datos de referencia con la distribución de los datos actuales.

Los indicadores registrados son:

- Variable evaluada.
- Valor estadístico de la prueba KS.
- Valor p obtenido.
- Cantidad total de variables evaluadas.
- Existencia o ausencia de Data Drift.
- Estado general del monitoreo.

Se considera que existe evidencia de Data Drift cuando el valor p es menor al nivel de significancia definido:

$p\text{-value} < 0.05$

1.4 Indicadores operativos del sistema

Además de las métricas del modelo, el sistema registra información operativa para facilitar la supervisión y la trazabilidad.

Los principales indicadores operativos son:

- Fecha y hora de ejecución.
- Inicio y finalización del proceso.
- Resultado del entrenamiento.
- Modelo almacenado correctamente.
- Registro exitoso en MLflow.
- Estado de las métricas.
- Estado de Data Drift.
- Nivel de alerta generado.
- Runbook recomendado.
- Errores o excepciones durante la ejecución.

1.5 Logs y trazas

Los eventos del sistema se almacenan en el archivo:

logs/monitor.log

Cada registro contiene información sobre la fecha, la hora, el nivel del evento y el mensaje asociado.

Los niveles utilizados son:

INFO: funcionamiento normal del sistema.

WARNING: desviación que requiere seguimiento.

CRITICAL: condición de alto impacto que requiere intervención inmediata.

ERROR: fallo durante la ejecución de algún componente.

El registro de logs permite reconstruir la secuencia de eventos, identificar fallas y facilitar el análisis de incidentes.

1.6 Frecuencia de monitoreo

En el prototipo desarrollado, los indicadores se actualizan cada vez que se ejecuta el sistema.

En un ambiente productivo, la frecuencia de monitoreo podría configurarse según las necesidades operativas, por ejemplo:

- Por cada lote de datos procesado.
- Cada hora.
- Diariamente.
- Después de cada actualización del modelo.
- Cuando se recibe nueva información.

2. Diseño de Alertas Inteligentes

El sistema incorpora un mecanismo de alertas inteligentes cuyo objetivo es identificar oportunamente posibles degradaciones en el desempeño del modelo de Machine Learning y facilitar la toma de decisiones por parte del responsable del sistema.

Las alertas fueron diseñadas considerando métricas críticas del modelo, principalmente el valor de Recall, debido a que representa la capacidad del modelo para identificar correctamente los casos positivos.

La clasificación de alertas implementada es la siguiente:

2.1 Niveles de alerta

Nivel INFO

Condición:

Recall mayor o igual a 0.85.

Descripción:

El modelo presenta un comportamiento estable y opera dentro de los parámetros esperados. No se requieren acciones correctivas.

Acción sugerida:

Continuar con el monitoreo normal.

Nivel WARNING

Condición:
Recall entre 0.75 y 0.84.

Descripción:
Se observa una disminución moderada en el desempeño del modelo. Aunque el sistema continúa operando, existe el riesgo de que el rendimiento siga deteriorándose si la tendencia continúa.

Acción sugerida:

- Revisar las métricas recientes.
- Verificar la calidad de los datos de entrada.
- Ejecutar el análisis de Data Drift.
- Incrementar la frecuencia del monitoreo.

Nivel CRITICAL

Condición:
Recall menor a 0.75.

Descripción:
El desempeño del modelo ha disminuido significativamente y podría afectar la calidad de las predicciones realizadas.

Acción sugerida:

- Analizar inmediatamente la causa del problema.
- Validar la calidad de los datos utilizados.
- Ejecutar nuevamente el entrenamiento del modelo.
- Evaluar la sustitución del modelo actualmente desplegado.

2.2 Priorización de incidentes

Las alertas se priorizan considerando el impacto operativo que podrían generar.

Nivel	Prioridad	Impacto
-----	-----	-----
INFO	Baja	Operación normal.
WARNING	Media	Riesgo potencial de degradación.
CRITICAL	Alta	Riesgo elevado de pérdida de desempeño del modelo.

2.3 Presupuesto de error (Error Budget)

Como referencia para este proyecto se estableció el siguiente presupuesto de error:

- Recall objetivo: ≥ 0.85
- Zona de observación: $0.75 - 0.84$
- Umbral crítico: < 0.75

Este esquema permite identificar cuándo el modelo comienza a consumir su margen aceptable de desempeño antes de requerir una intervención.

2.4 Integración con el sistema

Las alertas son generadas automáticamente por el módulo ``alerts.py``, el cual recibe las métricas calculadas por ``monitor.py``.

Posteriormente, el nivel de alerta es mostrado en la consola, registrado en el archivo de logs y utilizado por el módulo ``runbooks.py`` para recomendar las acciones correspondientes.

De esta manera, el sistema integra monitoreo, alertamiento y respuesta automatizada dentro de un mismo flujo de observabilidad.

3. Descripción de Dashboards

Con el propósito de facilitar el monitoreo y la supervisión del modelo de Machine Learning, se implementaron dos herramientas de visualización complementarias: un dashboard interactivo desarrollado con Streamlit y la plataforma MLflow para el seguimiento de experimentos.

Ambas herramientas permiten consultar información relevante sobre el comportamiento del modelo y facilitan la toma de decisiones durante su ciclo de vida.

3.1 Dashboard desarrollado con Streamlit

Se desarrolló una aplicación web utilizando Streamlit con el objetivo de simular escenarios de Data Drift de forma interactiva.

El dashboard permite al usuario:

- Seleccionar cualquiera de las variables numéricas del conjunto de datos.
- Modificar artificialmente la distribución de una variable mediante un porcentaje configurable.
- Ejecutar nuevamente la detección de Data Drift.
- Visualizar el estado generado por el detector (INFO, WARNING o CRITICAL).

- Conocer el número de variables evaluadas y si se detectó o no un cambio significativo en la distribución de los datos.

Esta interfaz facilita la validación del algoritmo de detección de Drift sin necesidad de modificar el código fuente, proporcionando una experiencia más intuitiva para el usuario.

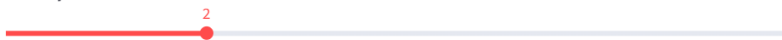
Monitor de Data Drift

Selecciona una variable y modifica sus valores para comprobar si el sistema detecta un cambio en la distribución.

Variable que deseas modificar

radius_mean

Porcentaje de modificación



Los valores de **radius_mean** se modificarán en 2%.

Evaluar Data Drift

Resultado

No se detectó Data Drift.

Estado: INFO

Variables evaluadas: 30

3.2 Dashboard de experimentos con MLflow

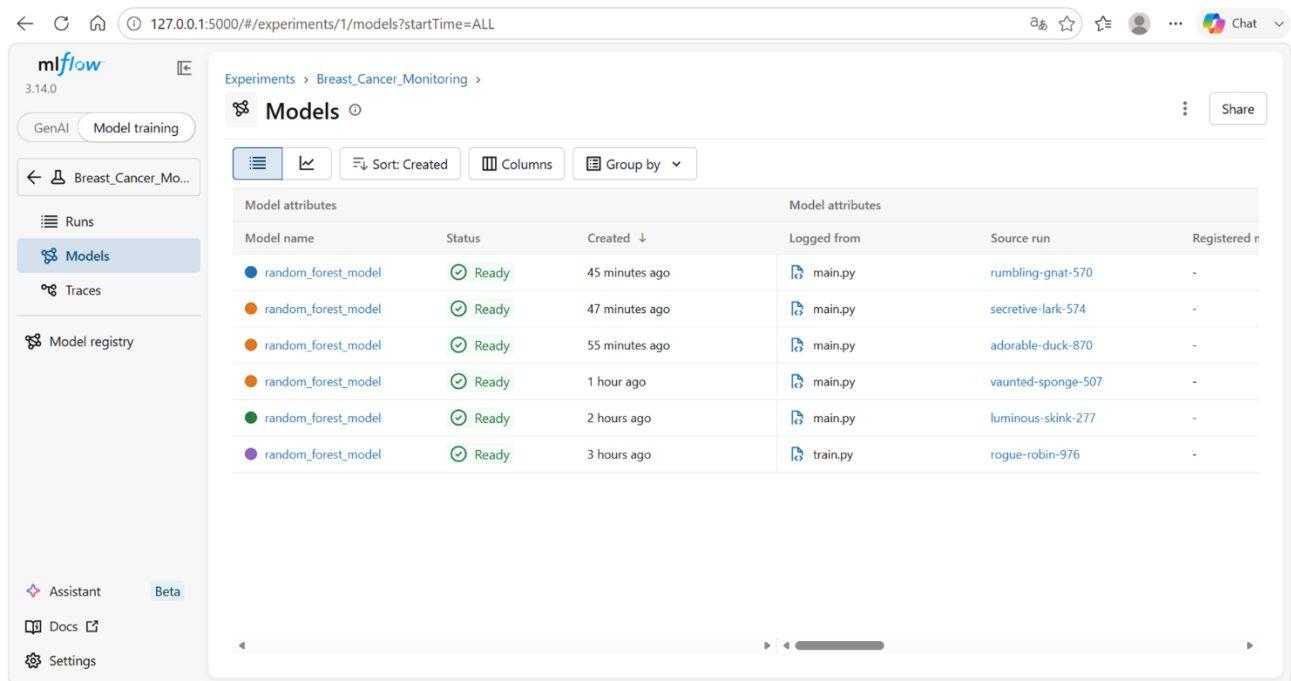
MLflow fue utilizado para registrar automáticamente los experimentos realizados durante el entrenamiento del modelo.

Cada ejecución almacena información relevante como:

- Parámetros utilizados durante el entrenamiento.
- Métricas de desempeño obtenidas.
- Fecha y hora de ejecución.
- Modelo generado.
- Historial de ejecuciones.

Esta información permite comparar diferentes versiones del modelo y mantener la trazabilidad del proceso de entrenamiento.

Figura 2. Dashboard de MLflow.



Experiments > Breast_Cancer_Monitoring >

Models

Sort: Created Columns Group by

Model attributes			Model attributes		
Model name	Status	Created ↓	Logged from	Source run	Registered name
random_forest_model	Ready	45 minutes ago	main.py	rumbling-gnat-570	-
random_forest_model	Ready	47 minutes ago	main.py	secretive-lark-574	-
random_forest_model	Ready	55 minutes ago	main.py	adorable-duck-870	-
random_forest_model	Ready	1 hour ago	main.py	vaunted-sponge-507	-
random_forest_model	Ready	2 hours ago	main.py	luminous-skink-277	-
random_forest_model	Ready	3 hours ago	train.py	rogue-robin-976	-

3.3 Beneficios de los dashboards

El uso conjunto de Streamlit y MLflow aporta diversas ventajas al proceso de monitoreo:

- Visualización centralizada de la información.
- Seguimiento histórico de experimentos.
- Detección rápida de anomalías.
- Validación visual de la presencia de Data Drift.
- Facilita la toma de decisiones basada en evidencia.
- Mejora la trazabilidad del ciclo de vida del modelo.

En un entorno productivo, este tipo de dashboards permite que los equipos de Ciencia de Datos, MLOps y Operaciones supervisen continuamente el estado de los modelos de Inteligencia Artificial y respondan oportunamente ante cualquier incidente.

4. Runbooks de Respuesta a Incidentes

Como parte del sistema de observabilidad se implementó un mecanismo de respuesta mediante Runbooks. Un Runbook es un procedimiento documentado que describe las acciones que deben ejecutarse cuando se detecta un incidente durante el monitoreo del modelo.

El objetivo de los Runbooks es estandarizar la respuesta ante eventos que puedan afectar el desempeño del modelo de Machine Learning, reduciendo los tiempos de reacción y facilitando la recuperación del servicio.

En este proyecto, los Runbooks son generados automáticamente por el módulo `runbooks.py`, tomando como entrada el nivel de alerta calculado por el sistema.

4.1 Runbook para estado INFO

Condición:

- Recall ≥ 0.85
- No se detecta Data Drift.

Descripción:

El modelo presenta un comportamiento estable y opera dentro de los parámetros esperados.

Acciones recomendadas:

- Continuar con el monitoreo periódico.
- Registrar la ejecución en los archivos de log.
- Mantener el modelo actualmente desplegado.
- No realizar acciones correctivas.

Resultado esperado:

Mantener la estabilidad y verificar que las métricas permanezcan dentro de los rangos establecidos.

4.2 Runbook para estado WARNING

Condición:

- Recall entre 0.75 y 0.84.
- Posible presencia de Data Drift.

Descripción:

Se detecta una degradación moderada en el desempeño del modelo. Aunque el sistema continúa funcionando, es recomendable investigar la causa antes de que el problema aumente.

Acciones recomendadas:

- Revisar las métricas registradas en MLflow.
- Analizar el archivo de logs.

- Verificar la calidad de los datos de entrada.
- Ejecutar nuevamente la prueba de Data Drift.
- Incrementar la frecuencia del monitoreo.

Resultado esperado:

Identificar oportunamente la causa del deterioro y evitar que el modelo continúe perdiendo desempeño.

4.3 Runbook para estado CRITICAL

Condición:

- Recall < 0.75.
- Evidencia importante de degradación del modelo.

Descripción:

El desempeño del modelo ha disminuido considerablemente y puede afectar la confiabilidad de las predicciones.

Acciones recomendadas:

- Detener temporalmente el despliegue del modelo si el impacto operativo lo justifica.
- Analizar la causa raíz del incidente.
- Validar la calidad y actualidad de los datos.
- Reentrenar el modelo utilizando información reciente.
- Comparar el nuevo modelo con la versión anterior mediante MLflow.
- Desplegar nuevamente el modelo únicamente si las métricas cumplen los criterios establecidos.

Resultado esperado:

Recuperar el desempeño del modelo y restablecer una operación confiable.

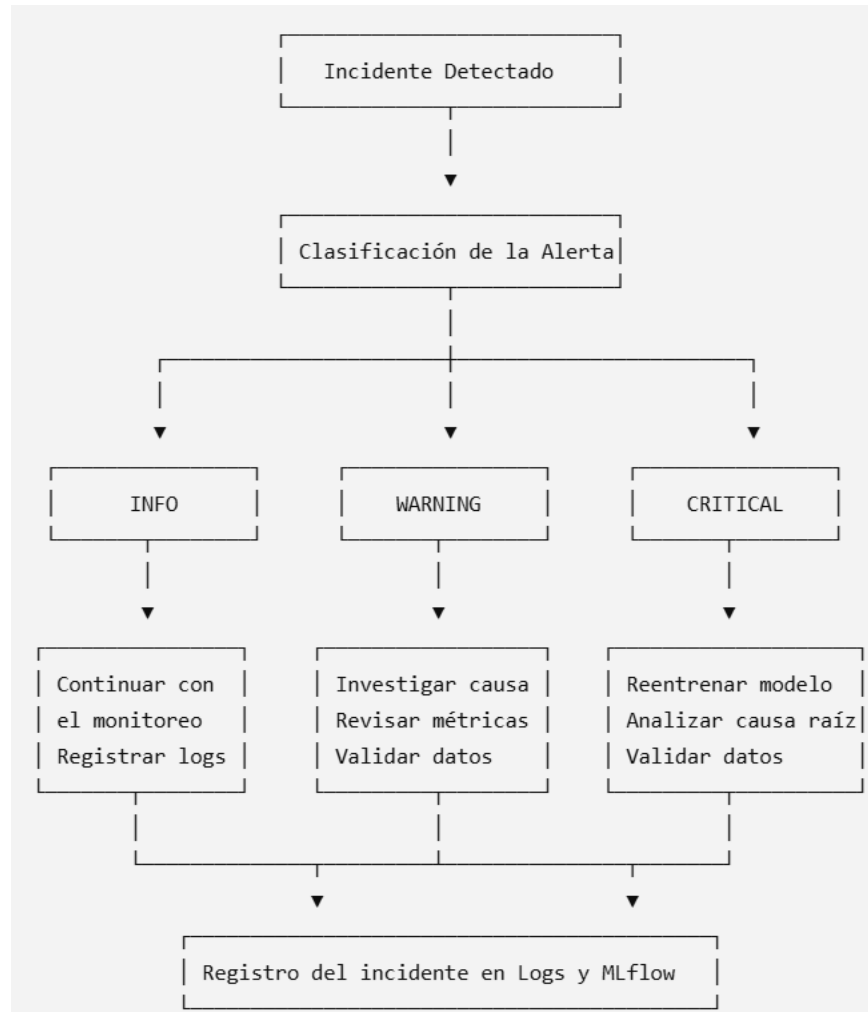
4.4 Beneficios del uso de Runbooks

La utilización de Runbooks aporta diversas ventajas al proceso de observabilidad:

- Reduce el tiempo de respuesta ante incidentes.
- Estandariza las acciones correctivas.

- Facilita la documentación de eventos.
- Disminuye la dependencia del conocimiento individual.
- Favorece la continuidad operativa.
- Mejora la trazabilidad de las intervenciones realizadas.

Aunque el proyecto corresponde a un entorno académico, el diseño de estos Runbooks está inspirado en prácticas utilizadas en equipos de MLOps para la administración y mantenimiento de modelos en producción.



5. Estrategias de Detección de Data Drift

Uno de los principales desafíos en los sistemas de Machine Learning es garantizar que el modelo continúe operando correctamente cuando recibe datos nuevos. Con el paso del tiempo, las características de los datos pueden cambiar debido a modificaciones en los procesos de negocio, variaciones en el entorno o cambios en la población de estudio.

Este fenómeno se conoce como **Data Drift** y puede provocar una disminución gradual en el desempeño del modelo, aun cuando este haya sido entrenado correctamente.

Con el propósito de identificar este tipo de situaciones, en el presente proyecto se implementó un mecanismo automático de detección de Data Drift.

5.1 Estrategia implementada

La estrategia consiste en comparar la distribución estadística del conjunto de datos utilizado durante el entrenamiento con un nuevo conjunto de datos de entrada.

Para realizar esta comparación se emplea la prueba estadística **Kolmogorov-Smirnov (KS Test)**, implementada mediante la biblioteca SciPy.

Esta prueba permite determinar si dos muestras provienen de la misma distribución estadística.

En el sistema desarrollado, cada variable numérica es evaluada de forma independiente.

El proceso consiste en:

- Seleccionar una variable del conjunto de datos.
- Generar una modificación controlada sobre sus valores mediante el dashboard de Streamlit.
- Comparar la distribución original contra la distribución modificada.
- Calcular el estadístico KS y su correspondiente valor p.
- Determinar si existe evidencia estadística de Data Drift.

5.2 Criterio de decisión

El criterio utilizado para determinar la existencia de Data Drift fue un nivel de significancia de:

$$\alpha = 0.05$$

La interpretación del resultado es la siguiente:

- Si el valor p es mayor o igual a 0.05, no existe evidencia suficiente para afirmar que las distribuciones son diferentes.
- Si el valor p es menor a 0.05, se considera que existe evidencia estadística de Data Drift.

Este criterio es ampliamente utilizado en pruebas de hipótesis y proporciona un equilibrio adecuado entre sensibilidad y confiabilidad.

5.3 Variables evaluadas

El detector analiza todas las variables numéricas presentes en el conjunto de datos, excluyendo aquellas que no aportan información relevante para el entrenamiento, como identificadores o columnas vacías.

Durante la ejecución del proyecto se evaluaron aproximadamente treinta variables correspondientes a características médicas utilizadas para la clasificación del cáncer de mama.

Cada variable es analizada individualmente, permitiendo identificar con precisión cuál presenta modificaciones significativas en su distribución.

5.4 Integración con el sistema

La detección de Data Drift fue implementada en el módulo ``drift.py``.

Este módulo recibe dos conjuntos de datos:

- Datos originales utilizados como referencia.
- Datos actuales o modificados.

Posteriormente ejecuta la prueba KS para cada variable y devuelve un resumen con la siguiente información:

- Estado del monitoreo.
- Existencia de Data Drift.
- Número de variables evaluadas.
- Variables que presentan cambios significativos (cuando aplica).

La información generada es utilizada posteriormente por los módulos de alertas y Runbooks para apoyar la toma de decisiones.

5.5 Validación mediante Streamlit

Con el objetivo de comprobar el funcionamiento del detector de Data Drift, se desarrolló un dashboard interactivo utilizando Streamlit.

Esta aplicación permite modificar artificialmente el porcentaje de una variable seleccionada y ejecutar nuevamente la prueba estadística.

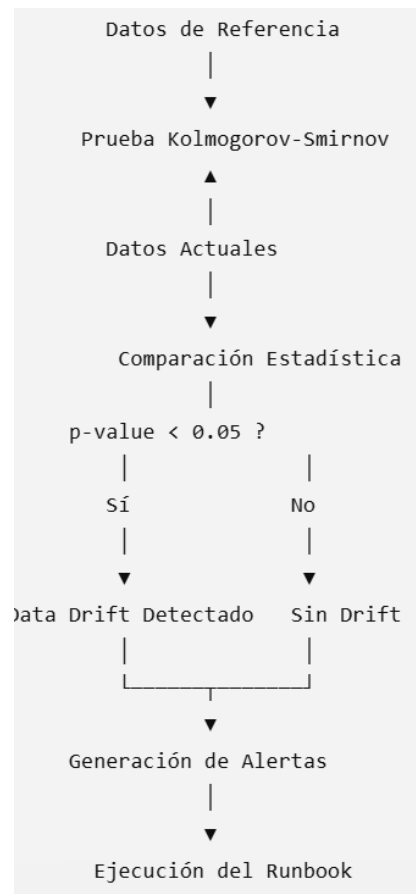
Gracias a esta funcionalidad fue posible observar cómo el sistema detecta cambios en la distribución de los datos conforme aumenta el porcentaje de modificación, validando así el comportamiento esperado del algoritmo implementado.

5.6 Beneficios de la estrategia implementada

La estrategia de detección de Data Drift desarrollada ofrece diversas ventajas:

- Permite identificar cambios en la distribución de los datos antes de que afecten significativamente el desempeño del modelo.
- Facilita el monitoreo continuo de modelos desplegados.
- Proporciona información útil para decidir cuándo reentrenar un modelo.
- Se integra fácilmente con sistemas de alertas y observabilidad.
- Puede adaptarse a escenarios reales de MLOps y monitoreo en producción.

Aunque el proyecto fue desarrollado con fines académicos, la estrategia implementada sigue principios utilizados en soluciones profesionales para el monitoreo del ciclo de vida de modelos de Inteligencia Artificial.



6. Documentación de la Respuesta Automatizada

Como parte del sistema de observabilidad desarrollado, se implementó un flujo de respuesta automatizada que integra el monitoreo del modelo, la generación de alertas, la detección de Data Drift y la recomendación de acciones mediante Runbooks.

El objetivo de esta automatización es reducir el tiempo de respuesta ante posibles incidentes y facilitar la supervisión continua del modelo sin depender completamente de la intervención manual.

6.1 Flujo de respuesta automatizada

El proceso implementado sigue la siguiente secuencia:

1. Se carga el conjunto de datos de referencia.
2. Se entrena el modelo Random Forest.
3. Se registran automáticamente los parámetros y métricas en MLflow.
4. El modelo entrenado se almacena mediante Joblib.
5. Se calculan las métricas de desempeño.
6. El sistema evalúa el estado del modelo.
7. Se ejecuta el detector de Data Drift.
8. Se genera el nivel de alerta correspondiente.
9. Se registra la información en el archivo de logs.
10. Se consulta el Runbook adecuado.
11. Se muestran los resultados al usuario mediante la consola y el dashboard de Streamlit.

Este flujo permite integrar todas las funcionalidades desarrolladas en un único proceso de observabilidad.

6.2 Componentes involucrados

La automatización se encuentra distribuida en diferentes módulos del proyecto.

Módulo	Responsabilidad
train.py	Entrenamiento del modelo y registro en MLflow.
monitor.py	Evaluación de métricas del modelo.
drift.py	Detección de Data Drift mediante KS Test.
alerts.py	Clasificación del nivel de alerta.
logger.py	Registro de eventos del sistema.
runbooks.py	Generación de recomendaciones de respuesta.
app.py	Interfaz gráfica para simulación mediante Streamlit.
main.py	Orquestación del flujo completo del sistema.

Cada componente cumple una función específica, permitiendo una arquitectura modular y de fácil mantenimiento.

6.3 Registro de eventos

Todos los eventos importantes son registrados automáticamente en el archivo:

logs/monitor.log

Entre los eventos registrados se incluyen:

- Inicio del proceso.
- Entrenamiento del modelo.
- Métricas obtenidas.
- Resultado del análisis de Data Drift.
- Nivel de alerta generado.
- Runbook recomendado.
- Finalización de la ejecución.

Este registro facilita la auditoría del sistema y el análisis posterior de incidentes.

6.4 Integración con MLflow

Durante cada ejecución, el sistema registra automáticamente la información del entrenamiento en MLflow.

La plataforma almacena:

- Parámetros utilizados.
- Métricas del modelo.
- Artefactos generados.
- Modelo entrenado.
- Historial de ejecuciones.

Esto garantiza la trazabilidad del ciclo de vida del modelo y permite comparar diferentes versiones cuando sea necesario realizar un reentrenamiento.

6.5 Beneficios de la automatización

La implementación de un flujo automatizado aporta múltiples ventajas:

- Reduce el tiempo de respuesta ante incidentes.
- Disminuye la posibilidad de errores manuales.
- Facilita el monitoreo continuo del modelo.

- Mejora la trazabilidad del ciclo de vida del modelo.
- Favorece la reproducibilidad de los experimentos.
- Simplifica el mantenimiento de soluciones de Machine Learning.

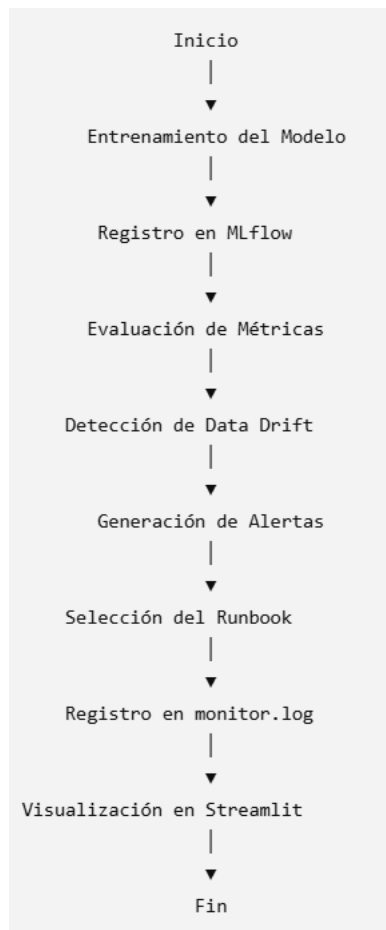
Aunque el sistema fue desarrollado con fines académicos, la arquitectura implementada sigue principios utilizados en plataformas modernas de MLOps y observabilidad para Inteligencia Artificial.

6.6 Conclusión del documento técnico

El sistema desarrollado demuestra cómo integrar herramientas de entrenamiento, monitoreo, observabilidad y respuesta automatizada dentro de un mismo flujo de trabajo.

La combinación de Random Forest, MLflow, Streamlit, Data Drift, alertas inteligentes y Runbooks permitió construir una solución que facilita la supervisión continua del modelo y proporciona mecanismos para detectar y atender posibles incidentes.

Este proyecto representa una aproximación práctica a las estrategias utilizadas actualmente para administrar modelos de Machine Learning en entornos de producción, reforzando los conceptos estudiados durante la materia de Gestión de Proyectos de Inteligencia Artificial.



7. Pipeline Automatizado e Integración Continua (CI)

Como parte de la evolución de la solución desarrollada, se incorporó un Pipeline Automatizado de Integración Continua (Continuous Integration, CI) utilizando GitHub Actions. Este componente permite automatizar la validación del proyecto cada vez que se realiza una actualización del código fuente en el repositorio, fortaleciendo la calidad, trazabilidad y mantenibilidad de la solución.

La automatización del pipeline representa una práctica fundamental dentro de las metodologías MLOps, ya que facilita la integración continua de cambios, reduce la posibilidad de errores manuales y permite validar el correcto funcionamiento de la aplicación antes de su despliegue.

7.1 Arquitectura del Pipeline

El pipeline desarrollado se encuentra definido mediante un archivo de configuración (ci.yml) ubicado en el directorio:

`.github/workflows/`

GitHub Actions interpreta este archivo y ejecuta automáticamente el flujo de trabajo cada vez que se realiza un Push o un Pull Request sobre la rama principal del proyecto.

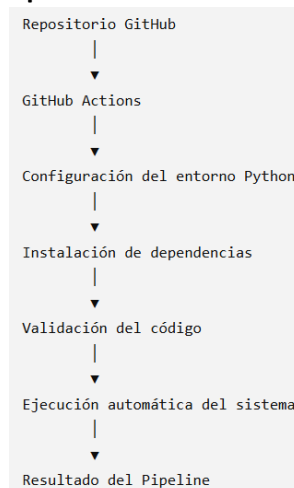
El flujo implementado sigue la siguiente secuencia:

1. Descarga automáticamente el repositorio desde GitHub.
2. Configura el entorno de ejecución con Python 3.13.
3. Instala las dependencias definidas en el archivo requirements.txt.
4. Verifica la sintaxis de los archivos principales del proyecto.
5. Ejecuta el sistema de monitoreo implementado en main.py.
6. Reporta el resultado de la ejecución dentro de GitHub Actions.

Este proceso garantiza que cada nueva versión del proyecto sea validada automáticamente antes de considerarse una versión funcional.

7.2 Flujo del Pipeline

El proceso automatizado implementado puede resumirse mediante el siguiente flujo:



7.3 Beneficios del Pipeline Automatizado

La incorporación de un pipeline automatizado aporta diversas ventajas al ciclo de vida del modelo de Machine Learning:

- Automatiza la validación del proyecto después de cada actualización del código.
- Reduce errores derivados de ejecuciones manuales.
- Garantiza que las dependencias del proyecto puedan instalarse correctamente.
- Verifica la integridad del código antes de su integración.
- Favorece la reproducibilidad de los experimentos y del entorno de ejecución.
- Incrementa la trazabilidad de las versiones del proyecto.
- Facilita la colaboración entre diferentes desarrolladores mediante un flujo de trabajo estandarizado.

En un entorno productivo, este tipo de automatización constituye uno de los pilares de las prácticas modernas de MLOps, al permitir que cada modificación del código sea evaluada de manera automática antes de incorporarse al proceso de operación.

7.4 Evidencia de Implementación

Como evidencia de la implementación del pipeline, se utilizó GitHub Actions para ejecutar automáticamente el flujo de integración continua. Durante las pruebas realizadas, el sistema ejecutó correctamente todas las etapas definidas en el archivo ci.yml, incluyendo la configuración del entorno, la instalación de dependencias, la validación del código fuente y la ejecución del sistema de monitoreo. La ejecución satisfactoria del pipeline demuestra que la solución desarrollada puede ser validada automáticamente mediante herramientas de integración continua, fortaleciendo la confiabilidad del proyecto y alineándolo con las prácticas recomendadas para el desarrollo y operación de soluciones de Inteligencia Artificial.

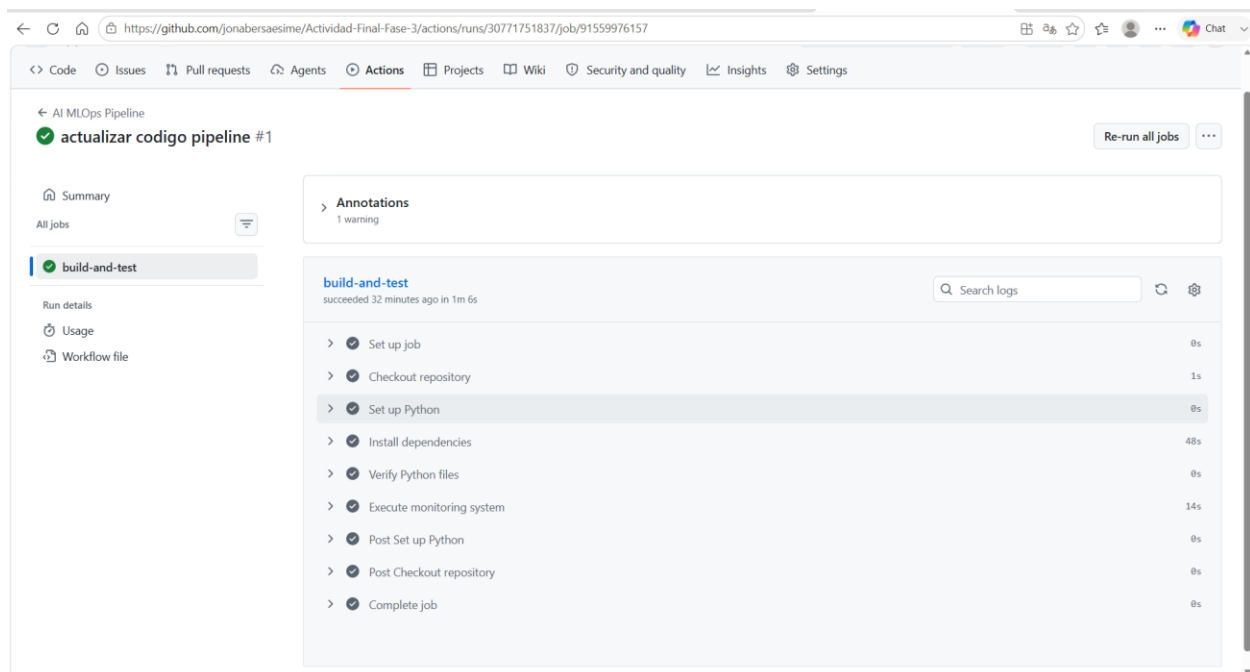


Figura 3. Ejecución exitosa del Pipeline Automatizado mediante GitHub Actions.

La figura muestra la ejecución satisfactoria del pipeline de Integración Continua (CI) implementado mediante GitHub Actions. Durante la ejecución se realizaron automáticamente la configuración del entorno de Python, la instalación de dependencias, la validación del código fuente y la ejecución del sistema de monitoreo, finalizando correctamente todas las etapas del flujo automatizado.

8. Estrategia GitOps y Control de Versiones

La solución desarrollada incorpora una estrategia de GitOps para administrar el ciclo de vida del código fuente, los artefactos del modelo y la configuración del pipeline automatizado. GitOps es una metodología que utiliza un sistema de control de versiones como fuente única de la verdad para gestionar aplicaciones e infraestructura, permitiendo que cada modificación realizada en el proyecto sea registrada, validada y rastreada de forma automática.

Para este proyecto se utilizó Git como sistema de control de versiones y GitHub como plataforma central para el almacenamiento del repositorio y la colaboración durante el desarrollo. Cada actualización del proyecto fue registrada mediante commits descriptivos, permitiendo mantener un historial completo de cambios y facilitando la recuperación de versiones anteriores en caso de ser necesario.

La integración entre GitHub y GitHub Actions permitió automatizar el proceso de validación del proyecto mediante un pipeline de Integración Continua (Continuous Integration, CI). Cada vez que se realiza una actualización sobre la rama principal del proyecto, GitHub ejecuta automáticamente el flujo definido en el archivo ci.yml, verificando la integridad del código antes de aceptar una nueva versión como válida.

8.1 Organización del Repositorio

El proyecto fue organizado siguiendo una estructura modular que facilita su mantenimiento, escalabilidad y reutilización. Cada componente del sistema fue separado en directorios específicos según su responsabilidad funcional, permitiendo una mejor administración del código y una mayor claridad durante el proceso de desarrollo.

La estructura principal del repositorio incluye los siguientes elementos:

Directorio / Archivo	Función
.github/workflows/	Configuración del Pipeline Automatizado mediante GitHub Actions.
src/	Módulos principales del sistema de monitoreo, alertas, detección de Data Drift y Runbooks.
models/	Modelo de Machine Learning serializado mediante Joblib.
mlruns/	Registro automático de experimentos realizado por MLflow.
logs/	Archivos de registro generados durante la operación del sistema.
data/	Conjunto de datos utilizado para entrenamiento y pruebas.
app.py	Dashboard interactivo desarrollado con Streamlit.
main.py	Punto de entrada del sistema de monitoreo.
requirements.txt	Dependencias necesarias para ejecutar la aplicación.
README.md	Documentación técnica del proyecto y guía de instalación.

8.2 Beneficios de la Estrategia GitOps

La implementación de GitOps aporta diversas ventajas durante el desarrollo y operación de soluciones de Inteligencia Artificial:

- Mantiene un historial completo de modificaciones realizadas al proyecto.
- Facilita el trabajo colaborativo mediante un repositorio centralizado.
- Garantiza la trazabilidad de cada cambio realizado en el código fuente.
- Permite automatizar procesos de validación mediante GitHub Actions.
- Favorece la reproducibilidad del entorno de desarrollo.
- Reduce errores asociados a cambios manuales.
- Incrementa la confiabilidad del proceso de integración continua.

Estas prácticas fortalecen la gobernanza del proyecto y facilitan la administración del ciclo de vida del modelo, alineándose con los principios fundamentales de MLOps, donde la automatización, la trazabilidad y la reproducibilidad constituyen elementos esenciales para el desarrollo de soluciones confiables de Inteligencia Artificial.

8.3 Evidencia de Implementación

Como evidencia de la implementación de la estrategia GitOps, el proyecto fue gestionado mediante un repositorio en GitHub, incorporando control de versiones, organización modular del código y un pipeline automatizado de Integración Continua. La Figura X muestra la estructura del proyecto dentro de Visual Studio Code, donde se observan los principales directorios utilizados para el desarrollo, así como la carpeta `.github/workflows`, encargada de almacenar la configuración del pipeline automatizado.

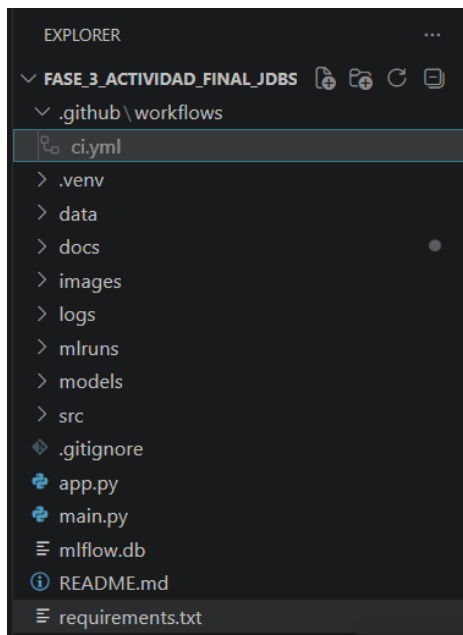


Figura 4. Estructura del proyecto en Visual Studio Code con organización modular y configuración de GitHub Actions.

9. Operación Continua de la Plataforma

La operación de la plataforma desarrollada contempla un flujo continuo de supervisión, validación y mantenimiento del modelo de Machine Learning, siguiendo principios de observabilidad y prácticas de MLOps. El objetivo principal consiste en garantizar que el modelo opere de manera confiable después de su entrenamiento, permitiendo detectar oportunamente cualquier degradación en su desempeño o cambios en la distribución de los datos.

La operación inicia con la ejecución del sistema de monitoreo implementado en `main.py`, el cual evalúa las métricas de desempeño del modelo, registra la información relevante mediante MLflow, ejecuta el detector de Data Drift y genera alertas cuando alguna condición supera los umbrales establecidos. Como complemento, el sistema consulta los Runbooks definidos para proporcionar recomendaciones de respuesta ante posibles incidentes.

De forma paralela, el Dashboard desarrollado con Streamlit permite visualizar el comportamiento del sistema en tiempo real y simular distintos escenarios de variación en los datos de entrada, facilitando la interpretación de resultados y el análisis del comportamiento del detector de Data Drift.

La incorporación del Pipeline Automatizado mediante GitHub Actions fortalece el proceso operativo al validar automáticamente el proyecto cada vez que se incorpora una nueva versión del código al repositorio. De esta forma, se garantiza que cualquier actualización sea verificada antes de formar parte de una versión estable del sistema.

9.1 Flujo Operativo

El funcionamiento general de la plataforma puede resumirse mediante el siguiente flujo operativo:

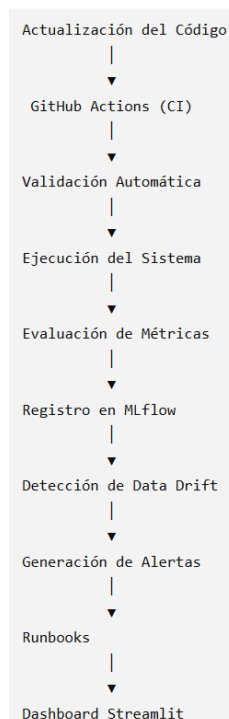


Figura 5. Flujo operativo de la plataforma MLOps desarrollada.

9.2 Estrategia de Mantenimiento

Para asegurar la operación continua de la plataforma se definieron las siguientes actividades de mantenimiento:

- Monitorear periódicamente las métricas de desempeño del modelo.
- Verificar la aparición de Data Drift mediante la prueba Kolmogorov-Smirnov.
- Analizar las alertas generadas automáticamente por el sistema.
- Consultar los Runbooks para determinar las acciones correctivas correspondientes.
- Registrar las ejecuciones y métricas mediante MLflow para mantener la trazabilidad del modelo.
- Actualizar el repositorio mediante Git y validar automáticamente los cambios utilizando GitHub Actions.
- Reentrenar el modelo cuando las métricas de desempeño o la detección de deriva indiquen una degradación significativa.

Estas actividades permiten mantener la estabilidad operativa del sistema y facilitan la administración del ciclo de vida del modelo conforme a las prácticas recomendadas de MLOps.

9.3 Resultado Operativo

La integración de GitHub, GitHub Actions, MLflow, Streamlit, el sistema de monitoreo, la detección de Data Drift y los Runbooks permitió construir una solución de Inteligencia Artificial preparada para operar bajo un esquema de supervisión continua. La plataforma implementada automatiza procesos clave de validación, monitoreo y control, mejorando la confiabilidad del sistema y proporcionando una base sólida para futuras implementaciones en entornos industriales.

Conclusión

El desarrollo de este proyecto permitió integrar los principales componentes de un sistema de observabilidad para modelos de Machine Learning, abordando aspectos que van más allá del entrenamiento del modelo y que resultan fundamentales para su operación en entornos reales. A través de la implementación de indicadores de desempeño, alertas inteligentes, dashboards de monitoreo,

detección de Data Drift y Runbooks de respuesta, fue posible construir un flujo de supervisión capaz de identificar oportunamente posibles anomalías y proponer acciones para su atención.

Asimismo, la incorporación de herramientas como MLflow para el registro de experimentos y Streamlit para la visualización interactiva fortaleció la trazabilidad y comprensión del comportamiento del modelo durante su ciclo de vida. Estas tecnologías representan prácticas ampliamente utilizadas en proyectos de MLOps, donde el monitoreo continuo y la automatización de procesos son elementos clave para garantizar la confiabilidad y el mantenimiento de soluciones basadas en Inteligencia Artificial.

Finalmente, este proyecto permitió aplicar de manera práctica los conocimientos adquiridos en la materia Gestión de Proyectos de Inteligencia Artificial, demostrando que la observabilidad constituye un componente esencial para asegurar que los modelos de Machine Learning mantengan un desempeño estable, sean capaces de adaptarse a cambios en los datos y proporcionen resultados confiables a lo largo del tiempo.

Conclusión

El desarrollo de este proyecto permitió integrar los principales componentes necesarios para la operación y supervisión de un modelo de Machine Learning bajo un enfoque de **MLOps**, abordando aspectos que van más allá del entrenamiento del modelo y que resultan fundamentales para su funcionamiento en entornos reales. A través de la implementación de indicadores de desempeño, alertas inteligentes, dashboards de monitoreo, detección de Data Drift y Runbooks de respuesta, fue posible construir un flujo de observabilidad capaz de identificar oportunamente posibles anomalías y proponer acciones para su atención.

Asimismo, la incorporación de herramientas como **MLflow** para el registro de experimentos, **Streamlit** para la visualización interactiva y **GitHub Actions** para la automatización del pipeline de Integración Continua fortaleció la trazabilidad, reproducibilidad y confiabilidad del sistema. La integración de estas tecnologías permitió automatizar procesos de validación, facilitar el control de versiones mediante Git y garantizar que cada actualización del proyecto sea verificada antes de formar parte de una versión estable.

De igual forma, la adopción de prácticas de **GitOps** y la organización modular del repositorio favorecieron la administración del ciclo de vida del proyecto, facilitando su mantenimiento, escalabilidad y evolución. Estas prácticas representan un componente esencial dentro de las metodologías modernas de desarrollo de soluciones de Inteligencia Artificial, donde la automatización y la colaboración constituyen elementos clave para asegurar la calidad del software.

Finalmente, este proyecto permitió aplicar de manera práctica los conocimientos adquiridos durante la materia **Gestión de Proyectos de Inteligencia Artificial**, integrando conceptos de Machine Learning, observabilidad, monitoreo, automatización y MLOps en una única solución funcional. Como resultado, se desarrolló una plataforma capaz de supervisar continuamente el comportamiento del modelo, detectar cambios en la distribución de los datos, automatizar la validación del proyecto mediante un pipeline de Integración Continua y proporcionar mecanismos de respuesta que fortalecen la confiabilidad y el mantenimiento del sistema durante todo su ciclo de vida.